

Curs 2

NLTK (Natural Language Toolkit) este o bibliotecă Python folosită pentru procesarea limbajului natural (NLP) și oferă: tokenizare, analiză gramaticală, etichetare, frecvențe de cuvinte, corpusuri lingvistice, lematizare etc.

```
import nltk
```

```
from nltk.tokenize import word_tokenize
```

```
from nltk.probability import FreqDist
```

word_tokenize (din nltk.tokenize) - împarte textul într-o listă de cuvinte și semne de punctuație și casă funcționeze, are nevoie de resursa **punkt** (reguli pre-antrenate de tokenizare).

FreqDist (din nltk.probability) - creează o distribuție de frecvență a tokenilor, numără de câte ori apare fiecare cuvânt.

metoda `.most_common(10)` extragi **cele mai frecvente 10 cuvinte**.

`nltk.download('punkt', quiet=True)` - descarcă resursa **punkt**, care este un model pre-antrenat pentru **tokenizare** (împărțirea textului în propoziții și cuvinte) și este necesar pentru funcții precum `word_tokenize` sau `sent_tokenize`.

Observatie: **pachetul punkt din NLTK** este utilizat pentru tokenizare, si este necesar descarea lui pentru funcționarea tokenizatorului. Dacă dorești să descarci pachetul punkt într-un anumit director (de exemplu, C:\Users\user), poți specifica calea la momentul descărcării.

```
nltk.download('punkt_tab', quiet=True)
```

Atenție: NLTK nu vine cu toate resursele preinstalate (cum ar fi modele antrenate, corpora de text sau tokenizatoare) dar folosește o funcție numită `download()` pentru a descărca aceste date **din internet** (de pe serverele NLTK) .

Sintaxa de bază: `nltk.download('numele_resursei')`.

Cu parametrul `quiet=True`, totul se întâmplă "în fundal", fără output pe ecran .

'punkt_tab' este numele unei **resurse specifice** din NLTK: un **model pre-antrenat pentru tokenizare** (împărțirea textului în tokeni, adică cuvinte sau propoziții).

PUNKT (de la "Punctuation Tokenizer") e un algoritm **nesupervizat** (nu are nevoie de etichetare manuală) care învață din date statistice să identifice **limitele propozițiilor** (unde se termină o propoziție bazat pe puncte, majuscule, abrevieri ca "Dr." sau "etc."). Ajută la **tokenizarea cuvintelor** (`word_tokenize`).

Versiunea **punkt_tab** este o **actualizare recentă** (din NLTK 3.8+), care înlocuiește vechea resursă 'punkt'.

Vechea versiune folosea "pickle-uri" (fișiere serializate) nesigure, care puteau avea vulnerabilități de securitate. **punkt_tab** e mai sigură și bazată pe tabele (tabular), de unde și numele.

Multe funcții NLTK, cum ar fi `nltk.word_tokenize(text)` sau `nltk.sent_tokenize(text)`, depind de acest model ca să "înțeleagă" structura textului.

from nltk.tokenize import RegexpTokenizer: Importă un tokenizer bazat pe expresii regulate (regex). Acesta împarte textul în "tokeni" (cuvinte sau subunități) folosind un pattern regex.

from nltk.probability import FreqDist: importă o clasă care calculează distribuția de frecvențe a tokenilor (cât de des apare fiecare cuvânt).

Crearea tokenizer-ului:

`tokenizer = RegexpTokenizer(r'\w+') → c` Creează un tokenizer care caută pattern-ul `\w+` (un sau mai multe caractere alfanumerice: litere, cifre, underscore). Asta înseamnă că va extrage cuvinte ignorând spațiile, punctuația etc. (ex: "Machine", "learning", "și" etc.).

Tokenizarea:

- `tokens = tokenizer.tokenize(text) →` împarte textul în listă de tokeni.

from nltk.tokenize import sent_tokenize, RegexpTokenizer

from nltk.probability import FreqDist

- **din punkt** → doar `sent_tokenize` (și implicit `word_tokenize`).
- **independent de punkt** → `RegexpTokenizer`, `FreqDist`.

NLP și expresii regex

Regex - sunt șabloane de text utilizate pentru căutarea, identificarea și manipularea șirurilor de caractere. În NLP, expresiile regulate contribuie la analiza și manipularea textelor cu aplicabilitate în:

1. **Căutare și identificare de modele** de modele specifice în texte și facilitează extragerea informațiilor relevante din text.
2. **Tokenizare** - procesul de divizare a unui text în unități semantice mai mici (token-uri). Cu expresiile regulate se pot defini regulile de tokenizare.
3. **Înlocuire și modificare** textului în funcție de anumite criterii.
4. **Validare și filtrare** în funcție de anumite criterii.
5. **Extragere de informații cu ajutorul unor** șabloane specifice, se pot extrage informații relevante din texte.
6. **Normalizare** textelor prin eliminarea variabilității și aducerea lor la un format standard (conversia a → A).
7. **Preprocesare:** pentru etape de preprocesare a datelor în NLP, textele sunt curățate de caractere speciale, simboluri etc.

Before we talk about processing words, we need to decide what counts as a word.

Let's start by looking at one particular corpus (plural corpora), a computer-readable collection of text or speech.

For example the Brown corpus is a million-word collection of samples from 500 written English texts from different genres (newspaper, fiction, non-fiction, academic, etc.), assembled at Brown University in 1963–64 (Kučera and Francis, 1967).

„Înainte să vorbim despre procesarea cuvintelor, trebuie să decidem ce anume considerăm a fi un cuvânt.

Să începem prin a analiza un corpus anume (plural: corpora), adică o colecție de text sau de vorbire, stocată într-o formă care poate fi citită de calculator.

De exemplu, corpusul *Brown* este o colecție de un milion de cuvinte, extrasă din 500 de texte scrise

în limba engleză din genuri diferite (ziare, ficțiune, non-ficțiune, texte academice etc.), alcătuită la Universitatea Brown în perioada 1963–1964 (Kučera și Francis, 1967).”

Pachete și biblioteci Python pentru manipularea expresiilor regulate în procesarea limbajului natural (NLP).

1. **re (modulul standard Python pentru expresii regulate):** permite căutarea, potrivirea și manipularea de șabloane în șirurile de caractere.

Aplicatia 1: o listă de tweet-uri trebuie curățate eliminând hashtag-urile, mențiunile, link-urile și alte caractere speciale.

```
import re
import csv
```

```
def curata_tweeturi(tweeturi):
    tweeturi_curate = []
```

```
    for tweet in tweeturi:
        # hashtag-urile
```

Pattern-ul `r'#\w+'`: `r` înseamnă "raw string" (evită escape-uri în Python), `#` – literalul caracter `#`, `\w+` – unul sau mai multe caractere "word" (litere, cifre, underscore: `a-z`, `A-Z`, `0-9`, `_`)

```
        tweet = re.sub(r'#\w+', '', tweet)
```

#Caută orice cuvânt care începe cu # (ex. `#AI`, `#Iot`) și îl înlocuiește cu șir gol → șterge hashtag-urile.

```
        # @tweets
```

```
        tweet = re.sub(r'@\w+', '', tweet)
```

```
        # link
```

```
        tweet = re.sub(r'http\S+', '', tweet)
```

Pattern-ul `r'http\S+'`: `http` – literal, `\S+` – unul sau mai multe caractere non-spațiu (orice în afară de spațiu: litere, cifre, simboluri), se elimină `http://x.com`

```
        # punctuație și caractere speciale
```

```
        tweet = re.sub(r'[\^\w\s]', '', tweet)
```

Pattern-ul `r'[\^\w\s]'`: `^` – clasă de caractere (ceea ce să caute), `^` în interior – negație (orice NU este...), `\w` – caractere word (litere, cifre, `_`), `\s` – spații (spațiu, tab, newline).

Elimină tot ce nu e literă, cifră, underscore sau spațiu: `!`, `?`, `..`, `..` etc

```
        # se adauga tweets curatat...
```

```
    tweet = re.sub(r'^\s+', '', tweet).strip() # pentru eliminare spații multiple
```

`\s+` = expresie regulată care înseamnă „unul sau mai multe spații albe” (spațiu, tab, newline).

Înlocuiește toate aparițiile cu un singur spațiu (' ').

Metoda `.strip()` aplicată pe un șir de caractere (string) șterge spațiile albe și caracterele de control de la începutul și sfârșitul textului.

```
    tweeturi_curate.append(tweet)
```

```
    return tweeturi_curate
```

```
def scrie_in_csv(tweeturi_curate, nume_fisier):
```

```
    with open(nume_fisier, 'w', newline='', encoding='utf-8') as csvfile:
```

```
        fieldnames = ['Tweet Curat']
```

```
        writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
```

```
        writer.writeheader()
```

```
        for tweet_curat in tweeturi_curate:
```

```
            writer.writerow({'Tweet Curat': tweet_curat})
```

```
# Exemplu de folosire
```

```
tweeturi_origin = [
```

```
    "tweet cu #hashtag interesant! @tweets",
```

```
"Verificati acest link: https://twitter.com/.com",  
"Mesaj despre #eveniment? @utilizator"
```

```
/
```

```
tweeturi_curate = curata_tweeturi(tweeturi_origin)
```

```
# Scrie datele curățate într-un fișier CSV  
scrie_in_csv(tweeturi_curate, 'c://2024/tweeturi_curate.csv')  
print("rezultat in CSV")
```

Aplicatia 2. Rezolvati aceeasi problema cu biblioteca NLTK

NLTK (Natural Language Toolkit): NLTK este o bibliotecă extinsă pentru procesarea limbajului natural și include și facilități pentru expresii regulate.

Rezolvare: Folosim funcționalități specifice NLTK precum **word_tokenize** pentru a separa cuvintele, **stopwords** pentru a elimina cuvintele comune și **isalnum** pentru a păstra doar caracterele alfanumerice. Acesta ar trebui să îndeplinească cerințele de curățare a textului folosind NLTK.

NLTK utilizează join pentru a reconstrui textul din cuvintele curate obținute prin tokenizare.

Discutii Stopwords:

- **Expresii Regulate (re):** nu are gestionată explicit eliminarea cuvintelor de oprire (stopwords).
- **NLTK:** Folosim stopwords din NLTK pentru a elimina cuvintele de oprire, adică cuvintele comune care nu adaugă adesea informații semnificative la sensul textului.

Reconstrucție text:

- **Expresii Regulate (re):** textul este reconstruit prin adăugarea cuvintelor curate la o listă și apoi concatenarea acestora într-un șir la sfârșit.
- **NLTK:** folosește join pentru a reconstrui textul din cuvintele curate obținute prin tokenizare.

```
import nltk  
from nltk.tokenize import word_tokenize  
from nltk.corpus import stopwords  
  
# Specifică calea dorită pentru descărcarea pachetului 'punkt'  
nltk.data.path.append(r'c://2025')  
  
# Descarcă pachetul 'punkt' și stopwords în calea specificată  
nltk.download('punkt', download_dir=r'c://2024')  
nltk.download('stopwords', download_dir=r'c://2024')  
  
def curata_tweeturi(tweeturi):  
    tweeturi_curate = []  
  
    stop_words = set(stopwords.words('english'))  
  
    for tweet in tweeturi:  
        # Tokenizare cuvinte  
        cuvinte = word_tokenize(tweet)  
  
        # Eliminare hashtag-uri, @tweets, link-uri și caractere speciale  
        cuvinte_curate = [cuvant for cuvant in cuvinte if cuvant not in stop_words and cuvant.isalnum()]
```

```

# Se reconstruiește textul din cuvintele curate
tweet_curat = ' '.join(cuvinte_curate)

# Se adaugă tweet-ul curat în lista rezultat
tweeturi_curate.append(tweet_curat)

return tweeturi_curate

def scrie_in_csv(tweeturi_curate, nume_fisier):
    with open(nume_fisier, 'w', newline='', encoding='utf-8') as csvfile:
        fieldnames = ['Tweet Curat']
        writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

        writer.writeheader()
        for tweet_curat in tweeturi_curate:
            writer.writerow({'Tweet Curat': tweet_curat})

# Exemplu de folosire
tweeturi_origin = [
    "tweet cu #hashtag interesant! @tweets",
    "Verificati acest link: https://twitter.com/.com",
    "Mesaj despre #eveniment? @utilizator"
]

tweeturi_curate = curata_tweeturi(tweeturi_origin)

# Scrie datele curățate într-un fișier CSV
scrie_in_csv(tweeturi_curate, 'c://2024/tweeturi_curate_nltk.csv')
print("Rezultatul a fost scris în fișierul CSV.")

```

TextBlob este o bibliotecă Python care ajută procesarea limbajului natural (NLP). Odată ce s-a creat un obiect TextBlob pe baza unui text, putem apela direct metodele și atributele sale pentru a analiza și manipula limbajul.

principale funcțiile din TextBlob:

- **words** - împarte textul în cuvinte (tokenizare la nivel de cuvinte).
Exemplu: "Artificial intelligence is transforming healthcare and education."
Rezultat: ['Artificial', 'intelligence', 'is', 'transforming', 'healthcare', 'and', 'education']
- **sentences** - împarte textul în propoziții.

Rezultat: [Sentence("Artificial intelligence is transforming healthcare and education.")]
- **noun_phrases** - extrage expresiile nominale (**frază cu substantive**), considerate cuvinte cheie.
Exemplu:
Rezultat: ['artificial intelligence', 'healthcare', 'education']
- **Tags** - Face Part-of-Speech tagging (atribuie roluri gramaticale fiecărui cuvânt).
Exemplu:
Rezultat: [('Artificial', 'JJ'), ('intelligence', 'NN'), ('is', 'VBZ'), ('transforming', 'VBG')]

<https://www.geeksforgeeks.org/nlp/nlp-part-of-speech-default-tagging/>

- Sentiment-analizează sentimentul textului și returnează polaritate și subiectivitate.
Polarity $\in [-1,1]$ (negativ-pozitiv), Subjectivity $\in [0,1]$ (obiectiv-subiectiv).
Exemplu:
Rezultat: Sentiment(polarity=0.1, subjectivity=0.3)

- translate(to='...')

Traduce textul în limba dorită.

Exemplu:

Rezultat: "Inteligența artificială transformă sănătatea și educația."

- correct()

Corectează greșeli de scriere.

Exemplu:

Text: "I havv goood speling" → Rezultat: "I have good spelling"